
QUANTEXCHANGE

High-Performance Electronic Trading & Market Simulation Engine

PROJECT DEEP DIVE

A complete, readable blueprint covering the problem, architecture, trading logic, modules, performance engineering, testing, implementation roadmap, and interview value.

OVERVIEW

1. Project at a Glance

Core idea: Build a realistic educational simulation of an electronic exchange. Instead of predicting stock prices, the system focuses on the infrastructure that receives, validates, prioritizes, matches, executes, records, and publishes trading activity.

Dimension	What the project demonstrates
Finance	Orders, bids/asks, spreads, liquidity, execution, positions, exposure and risk limits.
Algorithms	Price-time priority, order matching, partial fills, cancellation and efficient order-book operations.
Systems	In-memory state, event-driven architecture, concurrency, recovery and performance measurement.
Data	Trade/order persistence, streaming events, market analytics and synthetic workloads.
CS Fundamentals	C++, DSA, OOP, DBMS, networking, testing and performance engineering.

Why this is a strong flagship project

It connects financial-market concepts with serious computer-science problems. The strongest value is not the dashboard; it is the matching engine, order-book design, risk controls, event flow, correctness, benchmarks and engineering trade-offs.

THE CORE PROBLEM

2. Problem Statement

Problem statement: Modern electronic financial markets process large volumes of buy and sell orders within very short time intervals. Trading infrastructure must efficiently accept, validate, prioritize, match, execute, cancel and record these orders while maintaining correctness, low latency and reliable dissemination of market information.

Many student finance projects focus only on stock-price prediction or market-data dashboards. Those projects do not address the underlying computational problems of electronic trading infrastructure: order-book management, price-time priority, partial execution, pre-trade risk checks, high-throughput event processing, concurrency and performance optimization.

QuantExchange addresses this gap by simulating an electronic exchange. It accepts simulated trader orders, validates them, maintains order books for financial instruments, applies a defined matching algorithm, executes compatible orders, records trades, applies configurable risk controls and publishes market-data events. The system also measures throughput, latency and behavior under high-volume workloads.

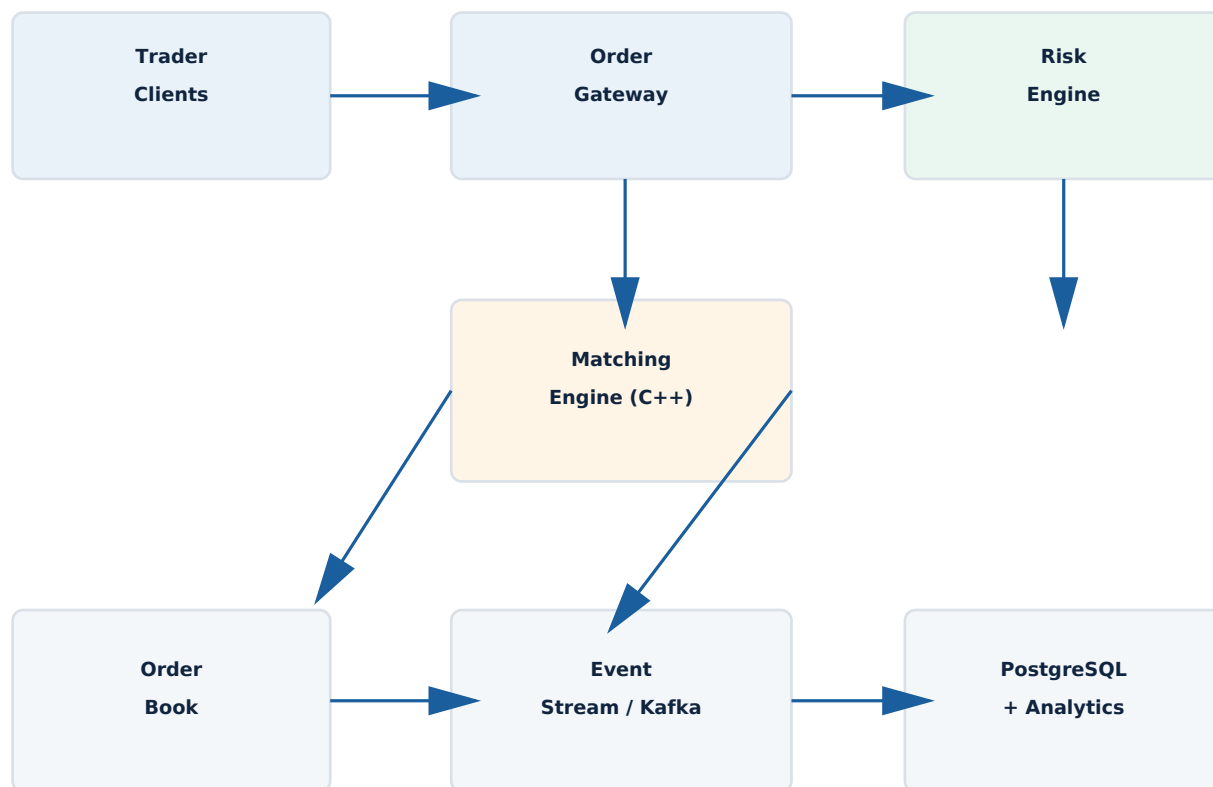
Project boundary

This is an educational exchange simulator. It does not execute real-money trades, connect to a live brokerage, or claim to reproduce every rule of a real-world exchange. The purpose is to model the computational and systems concepts clearly and safely.

HOW THE PIECES FIT TOGETHER

3. System Architecture

The system is best built as a layered, event-driven pipeline. Keep the matching engine simple and deterministic first, then add persistence, streaming, APIs and analytics around it.



High-level flow: Trader client -> Order Gateway -> Risk Engine -> Matching Engine -> Order Book / Trade Events -> Kafka or event stream -> Database and analytics -> Dashboard.

Recommended role of each layer:

- **Trader clients:** generate or submit simulated market/limit orders.
- **Order Gateway:** receives requests, assigns internal IDs and performs basic validation.
- **Risk Engine:** checks cash, holdings, order size, exposure and configured limits before execution.
- **Matching Engine:** the core C++ component that applies the trading rules and produces fills.
- **Order Book:** maintains active buy/sell orders by price level and time priority.
- **Event stream:** publishes accepted orders, executions, cancellations and market-data updates.
- **Persistence:** stores historical orders, trades, positions and audit information.
- **Dashboard:** visualizes the live order book, trades, market state and performance metrics.

WHAT THE ENGINE MUST UNDERSTAND

4. Core Trading Concepts

An order is a request to buy or sell a quantity of an instrument under defined conditions. The first implementation should support two essential order types: limit and market orders.

A limit order specifies a price constraint. A buy limit at Rs. 3,400 means the trader will not pay more than Rs. 3,400. A sell limit at Rs. 3,400 means the trader will not accept less than Rs. 3,400.

A market order requests immediate execution at currently available prices. It can consume multiple price levels until its requested quantity is filled or no more liquidity is available.

The order book stores active buy and sell orders. The highest buy price is the best bid; the lowest sell price is the best ask. Their difference is the spread.

Order book side	Priority rule	Typical data structure concept
BUY	Higher price first; then earlier time at equal price.	Ordered price levels + FIFO queue at each level.
SELL	Lower price first; then earlier time at equal price.	Ordered price levels + FIFO queue at each level.

The central matching rule is price-time priority: better prices get priority, and orders at the same price are normally handled in arrival order.

THE PROJECT BRAIN

5. Matching Engine: Step-by-Step

Example order book before an incoming order:

SELL
Price Quantity
3405 100
3404 50
3402 150

BUY
Price Quantity
3399 100
3398 80
3395 200

Incoming order: BUY 100 TCS @ Rs. 3402

The best ask is Rs. 3402. The buyer is willing to pay up to Rs. 3402, so the order can match. There are 150 shares available at that price; the buyer needs only 100.

Trade created: 100 @ 3402
Seller remaining: 50 @ 3402
Incoming buy remaining: 0

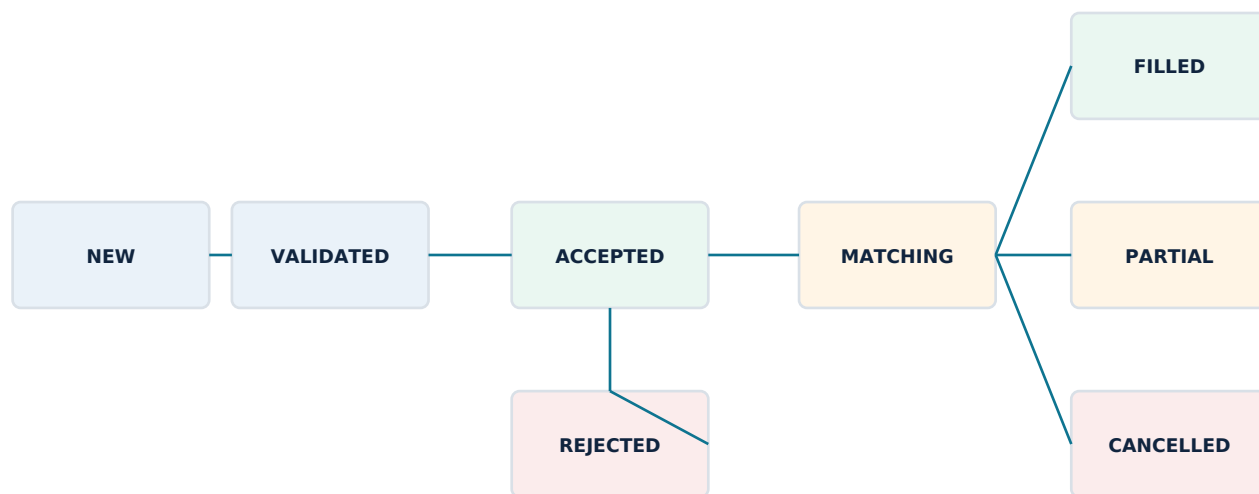
The same logic naturally handles partial fills. If a buyer requests 200 shares but only 80 are available at the current matching price, the engine executes 80 and keeps the remaining 120 active if the order rules permit resting.

Market orders should walk the book. For example, BUY 100 MARKET may consume 40 shares at Rs. 3,395, then 30 at Rs. 3,396, then 30 at Rs. 3,397, depending on the current sell-side liquidity.

STATE MACHINE

6. Order Lifecycle

Every order should have an explicit lifecycle. This makes the system auditable, testable and easier to debug.



Recommended states: NEW -> VALIDATED -> ACCEPTED -> MATCHING -> FILLED, with alternate paths to REJECTED or CANCELLED and a PARTIALLY_FILLED state before final execution.

Keep order status changes event-driven so that the same lifecycle can be used by the matching engine, persistence layer, monitoring system and audit trail.

WHAT YOU ACTUALLY IMPLEMENT

7. Core Modules

1. Order Gateway - Accept client requests, assign order IDs, normalize inputs and pass valid requests into the engine.
2. Validation - Check symbol, side, order type, quantity, price and request format before the order reaches risk logic.
3. Pre-Trade Risk Engine - Check cash, holdings, position limits, maximum order size, daily exposure and other configurable limits.
4. Order Book - Maintain buy/sell price levels and FIFO ordering within equal-price levels.
5. Matching Engine - Find compatible orders, execute fills, handle partial execution and update the book deterministically.
6. Trade Processor - Create trade records and update trader positions and realized state.
7. Market Data Publisher - Publish best bid/ask, last trade, volume, depth and other derived market events.
8. Persistence - Store orders, trades, positions, risk records and optionally the event log for recovery/replay.

A KEY DIFFERENTIATOR

8. Pre-Trade Risk Engine

The risk engine prevents invalid or policy-breaking simulated trades before they reach the matching process. This makes the project much closer to a financial infrastructure system than a simple order-entry app.

Rule	Example	Action
Cash check	100 shares x Rs. 3,400 requires Rs. 340,000.	Reject when available cash is insufficient.
Position limit	Maximum TCS position = 500 shares; current = 450; requested = 100.	Reject when projected position exceeds the configured limit.
Order size	Configured maximum order quantity = 10,000.	Reject larger requests.
Exposure	Total simulated market exposure exceeds trader limit.	Reject or route to a configurable risk state.
Daily loss	Simulated daily loss exceeds allowed threshold.	Block further orders until reset/override.

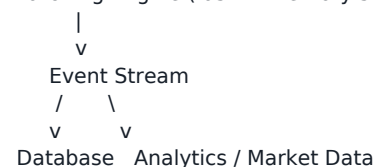
Make the risk rules configurable. In the report, clearly label thresholds as project settings rather than industry standards.

MAKING IT PRODUCTION-STYLE

9. Data, Events and Persistence

The in-memory matching engine should remain the fast core. Historical records can be persisted separately. This separation gives you a strong engineering story around latency, consistency and durability.

Matching Engine (fast in-memory state)



PostgreSQL can store orders, trades, positions, account state and audit records. Kafka or another event-streaming layer can carry accepted orders, fills, cancellations and market-data events to downstream consumers.

Example trade event:

```
{
  "event": "TRADE_EXECUTED",
  "symbol": "TCS",
  "price": 3400,
  "quantity": 200
}
```

An advanced recovery mode can persist an event log and replay events after restart to reconstruct the active order-book state. This introduces event sourcing and deterministic replay as optional advanced features.

PRACTICAL IMPLEMENTATION

10. Recommended Technology Stack

Layer	Recommended technology	Why
Matching engine	C++	Performance, explicit data structures, low-level control and strong DSA signal.
Analytics	Python + NumPy/pandas	Rapid analysis of fills, prices, volume and simulated market behavior.
Database	PostgreSQL	Reliable relational persistence for orders, trades and positions.
Cache / state	Redis (optional)	Fast state and dashboard-oriented caching where useful.
Event streaming	Kafka (advanced)	Event-driven architecture and decoupled consumers.
API	FastAPI or Node.js/Express	Expose simulation and analytics services.
Frontend	React	Interactive order-book, trade and performance dashboards.
Deployment	Docker	Reproducible local and cloud deployment.

Build order matters

Do not begin with React, Kafka, Docker and many services. First make the C++ matching engine correct. Then add persistence, APIs, streaming, dashboard and performance layers.

END-TO-END EXECUTION

11. What Happens When a Trader Clicks BUY?

Step	Component	Action
1	Client submits	BUY TCS, 100 shares, limit price Rs. 3,400.
2	Gateway	Receives the request and creates an internal order ID.
3	Validation	Checks syntax, symbol, quantity, price and order type.
4	Risk	Checks cash, holdings, position and configured exposure limits.
5	Book	Places the order into the TCS order book.
6	Match	Checks the best sell price and applies the price-time rule.
7	Execute	Creates one or more fills; may partially execute the order.
8	Update	Adjusts remaining quantity, trader positions and account state.
9	Publish	Emits trade/order/market-data events.
10	Persist	Stores the resulting records for history and audit.

WHAT MAKES IT STAND OUT

12. Performance Engineering

A student version becomes much more impressive when it measures its own behavior. Do not invent performance numbers. Benchmark your implementation

and report the real results.

Metric	Meaning	What to report
Throughput	Orders processed per second.	Orders/sec under defined workload.
Average latency	Mean time to process an order.	Average microseconds or milliseconds.
P95 latency	95% of requests complete below this value.	Tail latency under load.
P99 latency	99% of requests complete below this value.	Important for worst-case behavior.
Memory usage	Runtime memory consumed by the core process.	Peak and/or steady-state usage.
Error/rejection rate	Orders rejected or failed.	Percentage with reason categories.

Run progressively larger tests: 100,000 orders, 1 million orders and a higher-volume stress case. Compare data structures, allocation strategies or event-processing approaches only when you can explain the trade-off.

Advanced optimization topics include memory allocation behavior, object pools, lock contention, thread-safe queues, compact serialization and careful separation of serialized matching logic from parallel downstream analytics.

ADVANCED ENGINEERING DEPTH

13. Concurrency and Systems Design

Do not add multithreading just to claim that the project is parallel. The key systems question is: which operations can safely run concurrently while preserving deterministic order matching?

A sensible design is to keep the matching decision for a given instrument deterministic, while moving independent or downstream work - such as persistence, analytics or dashboard updates - into separate consumers or worker threads where appropriate.

Concern	Design question
Ordering	How do you establish a deterministic sequence for simultaneous orders?
Shared state	Which components own the order-book state?
Locks	Where is synchronization required and where can contention be avoided?
Failure	What happens if the database or event stream is temporarily unavailable?
Recovery	How is the active book rebuilt after a process restart?

OPTIONAL BUT VALUABLE

14. Market Simulation and Analytics

After the core exchange works, create simulated traders that generate different order-flow patterns. This allows you to test the engine without needing live market connectivity.

Trader model	Behavior
Market maker	Places buy and sell quotes around an estimated fair price.
Momentum trader	Buys or sells based on simulated short-term price movement.

Trader model	Behavior
Random trader	Generates stochastic orders for baseline load tests.
Institutional trader	Subdivides larger desired trades into smaller orders.
Risky trader	Attempts oversized orders or repeated limit breaches to test controls.

The analytics layer can calculate spread, volume, order imbalance, price movement, volatility, fill ratios and execution behavior across simulated market regimes.

CORRECTNESS BEFORE CLEVERNESS

15. Testing Strategy

Test	Input	Expected behavior
Full match	BUY 100 @ 100 vs SELL 100 @ 100	One 100-share trade.
Partial fill	BUY 100 @ 100 vs SELL 50 @ 100	50 executed, 50 remain.
No match	BUY 100 @ 99 vs SELL 100 @ 100	No trade; order rests if valid.
FIFO	Two buys at same price	Earlier order gets priority.
Cancellation	Cancel active order	Remaining quantity removed and status updated.
Insufficient cash	Buyer cannot fund requested position	Order rejected.
Position limit	Projected position exceeds configured limit	Order rejected.
Market order sweep	Market buy meets multiple sell levels	Consumes multiple price levels correctly.
Recovery	Restart after event-log checkpoint	Active book reconstructed consistently.

Add unit tests around the order book and matching engine first. Then add integration tests for risk, persistence and event streaming. Finally add load tests for throughput and tail latency.

BUILD IT IN VERSIONS

16. Implementation Roadmap

Version	Scope
V1 - Core engine	Order class, order book, limit orders, matching, trade generation.
V2 - Lifecycle	Market orders, cancellation, partial fills, status state machine.
V3 - Risk	Cash, holdings, exposure, position limits and order-size limits.
V4 - Persistence	PostgreSQL for orders, trades, positions and audit history.
V5 - Services	REST/WebSocket API and interactive dashboard.
V6 - Event-driven	Kafka/event stream, consumers and asynchronous analytics.
V7 - Performance	Benchmarks, stress tests, tail-latency reporting and optimization.
V8 - Advanced	Recovery/replay, simulated traders, circuit breakers and deeper analytics.

Recommended order

Build the matching engine correctly first. The dashboard and infrastructure should grow around a working, tested core.

HOW TO TALK ABOUT IT

17. Why This Project Is Interview-Strong

For a Goldman Sachs-style engineering interview, the strongest value is the number of genuine technical questions the project can support. Goldman's public engineering and assessment material emphasizes areas such as algorithms/data structures, software fundamentals, debugging, distributed systems, databases, scalable software and quantitative reasoning.

Likely question	What you should be able to explain
Why C++?	Performance, explicit data structures, memory behavior and systems control.
How does price-time priority work?	Price levels plus FIFO sequencing within equal-price levels.
What is partial execution?	How an incoming order consumes available liquidity and leaves a remainder.
Complexity?	The complexity of lookup, insert, delete and matching in your chosen data structures.
How do you prevent invalid trades?	Pre-trade risk checks before matching.
Why not write every event directly to SQL?	Latency and state-management trade-offs.
How do you handle restart?	Persistent event log and deterministic replay.
How did you measure performance?	Benchmark design, workload, latency and throughput methodology.
What happens under concurrency?	Ownership of book state, sequencing and synchronization strategy.

The best answer style is evidence-based: show your benchmark, show your test case, show the trade-off you considered and explain what you changed after profiling.

USE AFTER YOU ACTUALLY BUILD IT

18. Resume-Ready Description

Project title

QuantExchange - High-Performance Electronic Trading & Market Simulation Engine

Suggested resume bullets:

- Built a C++ electronic exchange simulator implementing price-time-priority order matching, limit/market orders, cancellation and partial fills.
- Implemented configurable pre-trade risk checks for cash, holdings, position limits and order-size/exposure constraints.
- Designed an event-driven architecture with persistent trade/order records, market-data publication and downstream analytics.

- Benchmarked throughput and P95/P99 latency under large synthetic order-flow workloads and profiled performance bottlenecks.

Only include measured numbers after you run the benchmarks yourself. Never put fabricated throughput or latency figures on the resume.

AVOID WEAKENING THE PROJECT

19. What Not to Do

- Do not make the dashboard the main feature and keep the backend trivial.
- Do not start with many microservices before the matching engine is correct.
- Do not add an LLM just to call the project "AI-powered".
- Do not claim real-world exchange behavior unless you explicitly implement and document the rule.
- Do not claim performance numbers without reproducible benchmarks.
- Do not copy a large codebase you cannot explain in an interview.

Golden rule

The project should be explainable from first principles. If an interviewer points to any important component, you should be able to explain what it does, why it exists, how it works and what trade-offs you made.

FOR THE GOLDMAN-ORIENTED CONTEXT

20. Reference Sources

Goldman Sachs Engineering Careers - <https://www.goldmansachs.com/careers/our-firm/engineering>

Goldman Sachs India Engineering Careers - <https://www.goldmansachs.com/worldwide/india/engineering-careers>

Goldman Sachs Engineering HackerRank Guide - <https://www.goldmansachs.com/careers/blog/guide-to-hackerrank>

Goldman Sachs India Hackathon -

<https://www.goldmansachs.com/careers/students/programs-and-internships/india/hackathon>

Goldman Sachs Student Careers - <https://www.goldmansachs.com/careers/students>

Note: The project design in this document is an educational portfolio blueprint. Specific technologies, architectural boundaries and rules can be adjusted based on implementation constraints and the level of the student project.